

Dash0 - Round 2: Technical Discovery Call

Stage: Second interview - technical discovery call **Previous round:** [dash0-senior-product-engineer.md](#)
Researched: 2026-05-18

Their ask: Think about 1-2 technically challenging projects you have worked on and be ready to discuss the technical details. Take a deep dive into our documentation and prepare to discuss your proposed approach to solving the kinds of problems we are currently working on.

This isn't a coding interview. They're looking for technical depth, genuine product curiosity, and the ability to connect your own experience to theirs. Have specific details ready - they'll probe past the summary.

Project 1: Live Technical Data Interpolation in CMS Copy

The problem

Volvo Cars' product pages are CMS-driven - editors write all the body copy. But a lot of that copy references car specs: electric range, charging times, power output, cargo capacity. Hard-coding those values creates two problems:

1. When a new model year ships with updated specs, someone has to manually hunt down every mention of every figure across every market - 40+ markets, multiple CMSes, no automated way to find them all.
2. The same car often has 4+ driveline variants with different specs. Either you write separate copy for each, or you fall back to vague language that avoids specifics altogether.

The solution

A placeholder syntax in the CMS that gets resolved at request time against live data from the federated GraphQL API:

Experience up to {HIGH:electricRange} of pure electric range on a single charge.

Editors write the copy once with placeholders. At render time, the server fetches the current specs, builds a structured data object, and `insertData()` runs a regex substitution over every string field in the CMS response - recursively, across the entire content tree.

Placeholder syntax

Placeholder	Resolves to
<code>{modelName}</code>	The car's display name
<code>{HIGH:electricRange}</code>	Highest electric range across all trims and drivetrains
<code>{LOW:acceleration}</code>	Lowest 0-100 time (fastest variant)
<code>{b4:kilowattsTotal}</code>	Power for a specific driveline code
<code>{b4:kilowattsTotal:exactValue}</code>	Raw number only
<code>{b4:kilowattsTotal:unit}</code>	Unit only (e.g. "kW")

Placeholder	Resolves to
<code>{b4:kilowattsTotal:label}</code>	Label string (e.g. "Max power")

`HIGH` and `LOW` are synthetic virtual drivelines computed at parse time - editors never need to know the actual driveline codes, which change between model years.

The interesting parts

Multi-source fallback per field

Electric range can come from 6 different API fields depending on the market's certification standard (WLTP, NEDC, CLTC, and regional variants). Each logical field defines a priority list:

```
electricRange: {
  keys: [
    'electricRangeCltcCombinedPer',
    'electricRangeCltcCombinedAer',
    'electricRangeWltpTotal',
    'electricRange',
    'electricRangeNedcCombined',
    'wltpElectricRangeCombinedRangeMax',
  ],
  defaultDrivelineCode: 'HIGH',
},
```

The parser takes the first non-null value. `batteryChargingDurationFast` has 9 fallback keys for different DC charging speeds (400kW down to 50kW) - whichever standard the car supports, the right value appears.

`HIGH` and `LOW` computed dynamically

`appendLowHighDrivelines()` iterates all real drivetrains and builds two synthetic entries representing the min and max value of each field. `{HIGH:electricRange}` always resolves even when driveline codes change between model years - editors write stable placeholders against a stable vocabulary.

Alternative driveline codes for renamed variants

For the EX90 Electric, some driveline codes were renamed between model years (`EN-L` → `DD-L`). Rather than requiring editors to update all their copy, there's a lookup table:

```
const ALTERNATIVE_DRIVELINE_CODES = {
  'ex90-electric': { 'EN-L': ['DD-L'], 'EV-E': ['DB-E'], 'EE-E': ['DA-E'] },
};
```

If the primary code isn't found, `insertData()` tries the alternatives before falling back to the raw placeholder text.

Recursive traversal across the content tree

`handleEntryResponse()` recurses through the entire CMS response - deeply nested arrays of objects - and applies the substitution to every string field. Editors can put placeholders anywhere without any per-component wiring.

Debug mode for editors

`?debug=true` wraps every substituted value in `$SOURCE_systemName:Car.fieldPath_$` markers, so editors can trace exactly where each value came from and verify data quality.

Feature-flag-gated field mapping

One luggage volume mapping change is being rolled out behind a LaunchDarkly flag (`feature.pdps.alternativeLuggageVolumeMapping`), which changes the priority order of which API fields are used for different seating configurations. The flag is evaluated server-side.

A subtle edge case

Cargo capacity varies by seating configuration - the same driveline can have a 5-seat and 7-seat variant with different boot volumes. The parser builds dynamic field keys like `cargoCapacity5Seats` and `cargoCapacity7Seats` at parse time, so editors can reference them separately: `{HIGH:cargoCapacity5Seats}`.

Result

Editors write copy once and it stays accurate across model year updates, market-specific certification standards, and driveline code changes. The same body text works across 40+ markets - units and number formatting come from the graph API, not the CMS.

Project 2: Data-Driven Playwright Testing Framework

The problem

Volvo Cars' PDPs render CMS-driven content. Any component on any page can change at any time when an editor publishes - no code deploy required. Traditional E2E tests miss these regressions because they test a snapshot of content, not the live system. And with 40+ markets and a growing list of car models, maintaining hand-written tests per combination wasn't sustainable.

The architecture

Content-aware test generation A generator script analyzes `.ct.ts` files (content type definitions from ContentStack) and auto-generates placeholder test classes. The mapping key is the content type `id`, not the class name - an important distinction that stops tests drifting when components are renamed in code.

CMS-driven test execution At test time, the framework calls `getPageContent()` to fetch live CMS data, maps each content type to its test class, and verifies against real content. If an editor adds a new component, it shows up as an explicit skipped test rather than silently passing.

Composable filtering Every test is automatically tagged with market, model, and page type. Tests can be filtered by any combination without manual maintenance:

```
E2E_COMPONENT_FILTER=subNavigation yarn test:e2e --grep "@uk.*@ex60-electric.*@overview"
```

Smart link verification Internal links (paths where `segments[1] === 'cars'`) are tested against the current environment. External links are tested against production. This prevents false positives in ephemeral PR environments where cross-app links would otherwise fail.

WCAG 2AA on every run `verifyAccessibility()` using axe-core runs on every page in every test. Accessibility isn't a separate suite - it's a gate on every commit.

`isImplemented = false` for tracking coverage debt Generated placeholders include this flag. The framework skips detailed verification for unimplemented components but reports them explicitly. You get partial coverage immediately on new components and can see exactly what's still outstanding.

Numbers

~32 mapped components, tested across multiple markets (UK, US, zh-CN, others) and car models. CI enforces a 900KB HTML size limit - pages over that fail the build.

Other Work Worth Mentioning

These aren't full stories but are worth having ready if they come up.

Granular CDN cache tagging The Next.js middleware sets three tiers of `Edge-Cache-Tag` on every response: page type, market, model, and market+model combination. This lets cache invalidation be targeted precisely - a spec change for the UK EX30 doesn't require flushing the entire global cache. CDN TTL is 7 days for pages, 30 days for assets. The design question was: how many tag dimensions do you actually need? Too few and you can't target anything; too many and every URL is unique and the cache is useless.

`useIdleCallback` with a feature-flag kill switch Below-the-fold data fetches (like the trim comparison table) are deferred to browser idle time using `requestIdleCallback` with a 500ms deadline, falling back to `setTimeout(fn, 0)` on Safari. The whole behaviour is gated behind a LaunchDarkly flag so it can be turned off instantly without a deploy if it causes problems. The motivation was reducing main thread contention during the initial render to improve INP scores.

URL slug to internal model ID translation Public URLs use consumer-friendly slugs (`ex30-electric` , `xc60-hybrid`) but the GraphQL API uses internal IDs (`ex30-bev` , `xc60-phev`). A small translation layer handles the mapping, plus special cases - `ex40-electric` maps to the `xc40` family for used car searches because both names refer to the same platform depending on market.

Talking Points on Dash0's Problems

These are angles to bring up naturally - not a script, just things you can speak to from actual experience.

Frontend observability and React Server Components

The `dash0-sdk-web` is the thing in the docs most relevant to what you've been building. Server-side instrumentation in Next.js is relatively straightforward - you have a persistent Node process, you can set up a tracer once and it's there for every request. The browser side is different: each page load is a fresh context, there's no persistent process, and with RSC the initial HTML is rendered server-side but large parts of the app then run client-side.

The specific question worth raising: how does trace context propagate across the RSC/client boundary? On the server you generate a trace with a specific trace ID. When the client hydrates and starts making its own requests, does the browser SDK know to attach to the existing trace or does it start a new one? If it starts a new one, you lose the connection between the server render and the client interaction - and that boundary is exactly where a lot of performance problems live.

This is a real open question, not a test. If they have a good answer, that's interesting. If they're still working on it, that's also interesting.

Agent0 and the UX of autonomous actions

At Volvo Cars we used LaunchDarkly to manage feature flags that could do meaningful things in production - like taking a product page offline for a specific market or car model entirely. That's a flag rule you can set in a UI that immediately affects what real users see.

What made it feel safe to use wasn't the technology, it was the UX design of the flag rule interface itself:

- You could always see the full history of who changed what and when
- Setting a rule and activating it were two separate actions - you had a moment to review before it went live
- Turning something back on was as easy as turning it off

Agent0 is doing something similar but the actor is AI rather than a person. The trust question is the same: how does someone know what the agent did, why it decided to do it, and how to undo it? I'd be curious how that's currently handled in the UI - whether there's a review step before the agent acts, or whether it's more a case of seeing what happened after.

Dashboard UX and what makes observability data actually useful

Running Prometheus and Grafana at home gave me a different perspective on observability tooling than I expected. The first thing I did was add as many metrics as possible - every stat I could scrape from OPNsense, TrueNAS, Home Assistant. Within a week I had 20+ graphs and I was looking at none of them.

What I ended up with after trimming back was a handful of panels I checked daily and one or two alerts that meant something. The rest was noise I'd convinced myself would be useful eventually.

The UI problem that's hardest to solve isn't showing the data - it's helping someone figure out which of their 200 metrics are actually worth watching. I'm curious whether that's something Dash0 is actively thinking about at the interface level, beyond just the cost angle.

Alerting - Sensitivity vs Noise

The core problem: how do you catch real issues quickly without firing on a single stray request?

What Dash0 actually provides

Trigger grace period - before an alert fires, the threshold condition must hold *continuously* for a set duration (options: 0s, 10s, 30s, 1m, 2m, 5m, 10m). A single bad request that spikes the error rate for 8 seconds and then clears never fires if the grace period is 30s. The condition has to persist, not just occur.

Keep-firing grace period - once an alert fires, it stays active for a minimum time after the condition clears. Prevents flapping, where an alert rapidly resolves and re-fires during an unstable period, generating noise of its own.

Enablement conditions - restrict when the rule evaluates at all. If a service gets 5 requests overnight and one fails, that's technically a 20% error rate but not an incident. An enablement condition gates evaluation on request volume being above a minimum threshold, so low-traffic periods don't produce meaningless alerts.

Two severity tiers - degraded and critical, each with its own threshold. Degraded can notify via Slack; critical wakes someone up. Not every alert needs the same response.

Evaluation interval - 1, 5, or 10 minutes. Controls how often the query runs, independent of the grace period.

The pattern that works

Rate-based threshold + trigger grace period + enablement condition.

Instead of alerting on a raw count of errors, write a PromQL query over a rolling window - e.g.

`rate(errors[5m])` - and threshold on that rate. A single error becomes a small blip that drops off as the window rolls forward. A real problem keeps the rate elevated through the grace period and fires. Since Dash0 is 100% PromQL compatible, any query logic from Grafana or Prometheus applies directly.

Where it still requires manual work

Multi-window alerting - alerting when error rate is elevated on both a short window (fast detection) and a long window (confidence it's real) - isn't a first-class feature as far as the docs show. It's doable with two separate PromQL queries, but requires manual setup per check. This is the pattern used in SLO-based alerting and is worth asking about.

Question to ask

"I've struggled in Grafana with the sensitivity/noise tradeoff - threshold too low and I get false positives from stray requests, too high and real problems slip through. I saw Dash0 has trigger grace periods and enablement conditions. In practice, do most users configure those manually per check, or is there a recommended pattern or template that handles the common cases well out of the box?"

Honest, specific, and the answer tells you how mature the alerting UX actually is.

Questions to Ask

Pick ones that come up naturally - don't recite all of them.

- **On the frontend SDK:** "How does `dash0-sdk-web` handle trace context across the RSC/client boundary in a Next.js app? That handoff is where I'd expect context to get dropped."
- **On Agent0:** "What does the current UX look like for Agent0 actions - does someone review before it acts, or is it more fire-and-observe right now?"
- **On the frontend team:** "What are the hardest UX problems on the product at the moment - not missing features, but places where the interface makes the data harder to use than it should be?"
- **On architecture:** "How are frontend decisions made - is there a dedicated frontend team, or is it more full-stack engineers owning slices end to end?"

Checklist

- ☐ Walk through `technical-data.utils.ts` cold - have `insertData()`, `appendLowHighDrivelines()`, and the multi-source fallback ready to explain
- ☐ Know the edge cases: alternative driveline codes (EX90), per-seating cargo capacity keys, feature-flagged luggage mapping
- ☐ Re-read `DATA_DRIVEN_TESTING.md` - know the numbers (32 components, multiple markets, 900KB CI limit)
- ☐ Read the Dash0 docs main page again - know the four pillars (metrics, logs, traces, resources) and what Agent0 is
- ☐ Have the LD flag UI story ready for the Agent0 question - audit trail, two-step activation, easy reversal

- ☐ Don't claim depth on ClickHouse, tail sampling, or Perses - fine to say you've read about them but haven't used them